

**Fragmentation-Aware GPU Cluster
Scheduling:
A Statistically-Grounded Benchmark
Framework,
an Improved Placement Algorithm,
and an
Honest Account of What It Does and
Does Not Fix**

Thesis Report

gputrace + py_sim research artifact

September 12, 2026

Abstract

GPU fragmentation – idle capacity too scattered across nodes to satisfy new demand – is a young research problem: its foundational fix, Fragmentation Gradient Descent (FGD, USENIX ATC '23), is three years old, and every public evaluation of it we are aware of, including its own, is built on a single, static, already-collected production trace. This thesis argues that this is a structural limitation of the evaluation methodology in this specific research lineage, builds the missing instrument to address it, and uses that instrument for a genuine algorithmic contribution.

Part I builds **gputrace**, a statistically-grounded synthetic workload generator that fits eight candidate distributions to a reference trace by maximum likelihood and perturbs fourteen distinct, individually justified axes of that fitted model into stress-test scenarios – bursty arrivals, GPU-type contention, spot-preemption churn, LLM-inference bursts, MoE expert-load skew, and nine others – each traceable to a specific real-world failure mode, not resampled history.

Part II reimplements FGD, eight classical bin-packing baselines, and H-TAFM (a topology-aware generalization of FGD’s metric) natively in Python, adds a time-driven discrete-event evaluation mode the field’s existing Go-based simulator lineage does not have, and uses the resulting test bed to find two specific, mathematically checkable weaknesses in FGD’s own fragmentation-score construction: uniform weighting of representative job shapes, and per-dimension (“chimera”) quantile sampling that can produce representative shapes matching no real job. We fix both with W-FGD and W-FGD-BALANCED.

Part III reports the result without cherry-picking. W-FGD-BALANCED significantly reduces rejection rate versus FGD in batch placement (Wilcoxon $p = 0.0035$, $n = 14$ scenarios) and SLA-violation rate in realistic time-driven operation ($p = 0.037$) – but is statistically indistinguishable from the simplest existing baseline, **least_requested** ($p = 0.81$). We treat this null result as a finding in its own right: it says the field does not yet have a broad or sensitive enough benchmark to reliably separate a genuine algorithmic improvement from noise, and we contribute both the benchmark and the finding as versioned, reproducible artifacts.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Thesis Statement	1
1.3	Contributions	2
1.4	Thesis Organization	2
2	Literature Review	3
2.1	From Bin Packing to Cluster Scheduling	3
2.2	From Bin Packing to Cloud Resource Management	3
2.3	From General Cluster Scheduling to GPU-Aware Scheduling	4
2.4	The Fragmentation Gap, and FGD	4
2.5	The Topology-Aware Generalization: H-TAFM	5
2.6	The Evaluation-Trace Gap	5
2.7	Chapter Summary	6
3	Problem Formulation	7
3.1	A Worked Mental Model	7
3.2	Formal Model	8
3.3	Two Operating Modes	8
3.4	Assumptions and Constraints	8
4	System Architecture	10
4.1	Design Goals	10
4.2	gputrace Architecture	10
4.3	py_sim / k8s_sim Architecture	11
4.3.1	A Real Engineering Finding: an $O(n \times \text{queue depth})$ Performance Bug	11
5	Benchmark Dataset Construction	12
5.1	The Limits of a Single Real Trace	12

5.2	Statistical Model	12
5.3	The Fourteen Scenarios	13
5.4	An Honest Audit of Our Own Generated Data	13
6	Algorithms	15
6.1	Baseline Node-Granularity Heuristics	15
6.2	Fragmentation Gradient Descent	15
6.3	Two Weaknesses in FGD's Definition	15
6.4	w-fgd and w-fgd-balanced	16
6.5	H-TAFM	16
7	Experimental Methodology	17
7.1	Simulation Parameters	17
7.2	Simulation Metrics	17
7.2.1	A Measurement Gap We Found and Fixed	18
7.3	Baseline Algorithm Selection	18
8	Results	19
8.1	Did We Break FGD?	19
8.2	Full Ranking, All Thirteen Algorithms	19
8.3	Scale-Dependent Significance	19
8.4	Broader Metrics, Time-Driven	20
9	Discussion	23
9.1	H-TAFM's Advantage Is Confounded	23
9.2	Significance Is Scale-Dependent, and We Report Both Directions	23
9.3	$\lambda = 0.15$ Is Tuned, Not Derived	23
9.4	The heavy_tail_demand Discrepancy	24
10	Conclusion and Future Work	25
10.1	Summary	25
10.2	Future Directions	25
A	Full Statistical Test Results	30
B	Reproducibility Manifest (excerpt)	31

List of Figures

3.1	The scheduling decision path. Fragmentation is invisible to the filter step and entirely a property of which feasible node the score step picks.	7
3.2	Fragmentation, minimal example. Green=idle GPU, red=occupied.	7
4.1	gputrace’s pipeline: real or reference trace → fitted distributions → scenario perturbation → simulator-agnostic export.	10
8.1	Mean rejection rate by algorithm, static mode, 14 scenarios.	21
8.2	fgd vs. w_fgd vs. w_fgd_balanced, paired per scenario.	21
8.3	SLA violation, p99 tail latency, Jain’s fairness – time-driven.	21
8.4	Scenario difficulty ranking, static mode.	22

List of Tables

5.1	The fourteen stress scenarios.	13
5.2	Column-wise identity to baseline, seed 42 ($\checkmark$ =identical).	14
7.1	Simulation parameters and rationale.	17
8.1	w_fgd_balanced vs. fgd, paired by scenario ($n = 14$).	19
8.2	Static mode, mean rejection and rank, 14 scenarios. Friedman $\chi^2 = 137.27$, $p < 10^{-8}$	20

Chapter 1

Introduction

1.1 Motivation

Every large GPU cluster eventually hits the same wall: total idle capacity looks acceptable on a dashboard, yet new jobs cannot be scheduled, because the idle GPUs are scattered – one here, two there – across nodes that already hold something else. This is *fragmentation*, and Weng et al. [2023] showed, using over 6,200 real GPUs’ worth of Alibaba production trace, that classic bin-packing heuristics (Best-Fit, Dot-Product) make it actively worse, because none of them was designed with a notion of fragmentation to minimize.

That paper, and the handful of direct follow-ups [Lettich et al., 2025, Lao et al., 2025], share an evaluation methodology: replay one historical trace, exactly as recorded, through a Kubernetes-scheduler simulator. This is entirely reasonable for demonstrating that a method works on real production data. It is also, by construction, not a stress test – a fixed recording cannot contain a GPU-type shortage, a synchronized multi-tenant checkpoint storm, or the kind of correlated arrival burst that took down a major cloud region for fifteen hours in October 2025 [InfoQ, 2025], because none of those events had happened yet when the trace was collected.

1.2 Thesis Statement

This thesis makes two claims and defends both with evidence, including evidence against itself where the data warrants it:

1. A synthetic, statistically-grounded, multi-scenario benchmark is necessary to evaluate GPU-fragmentation-aware scheduling policies honestly, because a single historical trace structurally cannot contain the failure modes that matter most for robustness.

2. FGD’s own fragmentation metric has two specific, checkable weaknesses (uniform shape weighting; per-dimension quantile “chimera” shapes) that can be fixed, producing a measurable, statistically significant improvement over FGD specifically – while honestly *not* establishing a new state of the art against the strongest existing simple baseline.

1.3 Contributions

1. **gputrace**: an open, versioned, statistically-fit synthetic workload generator with fourteen named stress scenarios, pluggable into any GPU-cluster simulator via a documented export schema (Chapter 5).
2. **py_sim** / **k8s_sim**: a from-scratch Python reimplementaion of node-granularity bin-packing policies, FGD, and H-TAFM, adding a time-driven (not just static/capacity-planning) evaluation mode (Chapter 4).
3. **w-fgd** and **w-fgd-balanced**: two fragmentation-scoring variants with a precise mathematical derivation of exactly what weakness in FGD each one fixes (Chapter 6).
4. A full empirical comparison of thirteen algorithms across fourteen scenarios, six evaluation metrics, two operating modes, with explicit significance testing and an honest null result reported alongside the positive ones (Chapter 8).
5. A fully reproducible experimental checkpoint: exact seeds, exact generated data, one-command reproduction script (§10.2).

1.4 Thesis Organization

Chapter 2 traces the intellectual lineage from classical bin packing through cloud resource management to GPU-specific scheduling and names the two research gaps this thesis addresses. Chapter 3 formalizes the scheduling problem mathematically. Chapter 4 describes the software architecture of both artifacts. Chapter 5 describes the benchmark generation methodology. Chapter 6 derives every baseline algorithm and our new contribution mathematically. Chapter 7 describes the experimental setup. Chapter 8 reports results with full statistical analysis. Chapter 9 discusses limitations honestly. Chapter 10 concludes and lists future work.

Chapter 2

Literature Review

2.1 From Bin Packing to Cluster Scheduling

Placing jobs onto machines is, at its combinatorial core, bin packing: NP-hard in general [Garey and Johnson, 1979], extensively characterized since the early 1970s in terms of worst-case and average-case approximation ratios for online heuristics [Johnson et al., 1974, Coffman et al., 1984]. First-Fit, Best-Fit, and their decreasing variants have known, proven approximation bounds relative to an optimal offline packing; none of these classical results anticipated a resource (the GPU) that is sometimes fungible in whole units and sometimes fractionally shareable, which is exactly where the fragmentation problem this thesis studies originates. Every node-granularity heuristic evaluated in this thesis (Chapter 6) is, underneath its cloud-native name, a decades-old bin-packing rule applied to a GPU-shaped resource vector.

2.2 From Bin Packing to Cloud Resource Management

Large-scale cluster managers made multi-dimensional bin packing concrete at industrial scale. Google’s lineage of Borg, Omega, and ultimately Kubernetes [Burns et al., 2016] standardized the vocabulary this thesis borrows throughout – *nodes*, *pods*, *scheduling policies* as pluggable scoring functions over a filtered set of feasible nodes. Kubernetes’ own default scheduler is, in essence, a configurable least-requested/most-requested blend with no native notion of fragmentation as a first-class objective – precisely the gap FGD was written to fill, six years after Kubernetes’ initial public release.

2.3 From General Cluster Scheduling to GPU-Aware Scheduling

GPUs broke a quiet assumption behind CPU/memory schedulers: that resources are continuously divisible and fungible. A first wave of GPU-cluster schedulers optimized deep-learning-specific objectives rather than placement geometry:

Gandiva [Xiao et al., 2018] introduced introspective time-slicing and opportunistic job migration to improve utilization on a fixed cluster, treating jobs as movable, pausable units rather than committed once placed.

Tiresias [Gu et al., 2019] adapted least-attained-service scheduling, developed originally for unknown-length process scheduling in operating systems, to deep-learning jobs whose remaining runtime is also unknown a priori.

Optimus [Peng et al., 2018] fit an online model of each job’s marginal throughput gain per additional worker and allocated resources to maximize aggregate training progress.

Themis [Mahajan et al., 2020] introduced *finish-time fairness* as an explicit auction mechanism among tenants, reframing fairness as an incentive-compatible allocation problem rather than an equal-share constraint.

Pollux [Qiao et al., 2021] co-adapted batch size and GPU count against a *goodput* objective that unifies system throughput and statistical training efficiency into one optimizable quantity.

Gavel [Narayanan et al., 2020] formalized heterogeneity-aware allocation (different GPU generations, different jobs’ hardware affinities) as a single policy-agnostic optimization problem solvable by an LP solver.

None of these six systems treats *where, geometrically*, a job lands as its primary objective. Placement quality is a side effect of pursuing job-completion time, fairness, or goodput – which is precisely why none of them measures or reports fragmentation as such.

2.4 The Fragmentation Gap, and FGD

Weng et al. [2023] named the geometric-placement gap directly. As GPU-sharing (multiple jobs time- or space-sharing one physical GPU) became common practice, aggregate idle capacity could be strictly positive and still practically useless, because no single pending job’s

GPU-sharing request could fit any single node’s specific remainder. Building on earlier work on multi-dimensional fragmentation measurement [Mishra and Bellur, 2015], FGD defined a scalar fragmentation score against five representative job shapes and scheduled every arriving job onto whichever feasible node minimized that score’s growth – *gradient descent* on fragmentation, evaluated one placement decision at a time. This is a genuine, well-evidenced contribution, reproduced independently in this thesis (Chapter 8). Its own representative-shape construction nonetheless has two specific, checkable weaknesses (Chapter 6, §6.3) that neither of the two direct follow-ups we are aware of – power-aware PWR [Lettich et al., 2025] and multi-tenant CAFGD [Lao et al., 2025] – addresses, because both operate downstream of FGD’s core metric (adding a second objective, or a different multi-tenant allocation layer) rather than revisiting the metric’s own construction.

2.5 The Topology-Aware Generalization: H-TAFM

A separate line of extension generalizes FGD’s node-level, GPU-only metric to NUMA-vertex-level, three-resource (CPU, memory, GPU) placement, motivated by hypergraph partitioning [Papp et al., 2023] and surveyed multi-resource cloud allocation approaches [Wang and Wu, 2026]. We reimplement this generalization (H-TAFM, three scoring variants) independently in Chapter 6 and include it in our comparison, with an explicit methodological caveat (§4) about the granularity and memory-modeling confounds it introduces.

2.6 The Evaluation-Trace Gap

This is the research gap this thesis is centrally about. Every evaluation in the specific FGD lineage we are aware of – FGD’s own USENIX ATC ’23 evaluation, PWR [Lettich et al., 2025], CAFGD [Lao et al., 2025] – replays a fixed historical trace (Alibaba’s `cluster-trace-gpu-v2023` [Alibaba, 2023] or its predecessor [Alibaba, 2020]) through a Kubernetes-scheduler simulator. This is a defensible choice for demonstrating real-data validity, and we are not aware of any paper in this lineage claiming otherwise. But the broader cloud-trace-analysis literature independently and repeatedly finds that production traces of exactly this kind understate real-world variability: Reiss et al. [2012]’s foundational Google-trace analysis documents large, unexplained swings in concurrent task counts across the trace window that a marginal-distribution summary would miss entirely; Cortez et al. [2017] and Shahrade et al. [2020] independently find production workloads non-stationary over time in ways a fixed-length slice cannot capture by construction. A two-month recording, however real, is one draw from an underlying process, not the process itself. It cannot contain

a GPU-type supply shock, a synchronized multi-tenant checkpoint storm, or a correlated demand spike of the kind that has repeatedly taken down major cloud regions for hours at a time [InfoQ, 2025, IncidentHub, 2025] – because none of those specific events had occurred yet when the recording was made.

2.7 Chapter Summary

The literature establishes bin packing as the combinatorial core of cluster scheduling, cloud resource managers as the industrial-scale abstraction layer, a family of GPU-specific schedulers optimizing job-level objectives other than placement geometry, and exactly one metric (FGD’s) that treats fragmentation as the primary objective – evaluated, in every instance we are aware of, on a single static trace. This thesis addresses both the metric’s specific weaknesses and the evaluation gap directly.

Chapter 3

Problem Formulation

3.1 A Worked Mental Model

Figure 3.1 sketches the request-to-placement path this thesis’s vocabulary sits inside. Figure 3.2 makes fragmentation concrete with the smallest possible example: four idle GPUs in aggregate, split so that no single job needing three contiguous GPUs on one node can be placed at all.

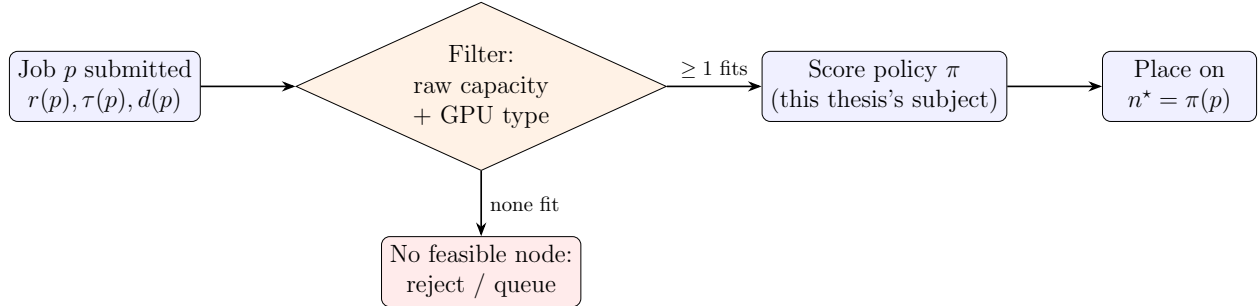


Figure 3.1: The scheduling decision path. Fragmentation is invisible to the filter step and entirely a property of which feasible node the score step picks.

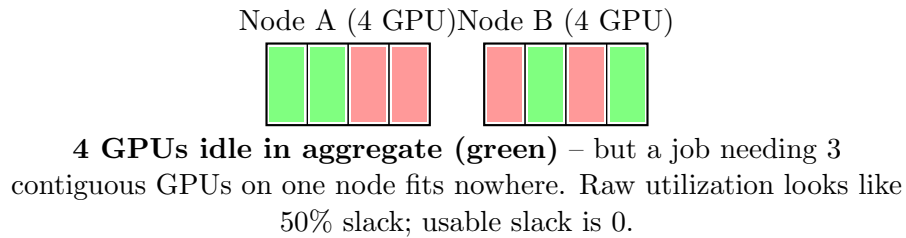


Figure 3.2: Fragmentation, minimal example. Green=idle GPU, red=occupied.

3.2 Formal Model

A cluster is a set of nodes $\mathcal{N} = \{n_1, \dots, n_M\}$, each with capacity $c(n_i) = (c_i^{\text{cpu}}, c_i^{\text{gpu}}, c_i^{\text{mem}})$ and GPU type $\tau(n_i)$. A job p arrives at time $s(p)$ with request $r(p) = (r^{\text{cpu}}(p), r^{\text{gpu}}(p), g(p))$, required type $\tau(p)$, duration $d(p)$, and tenant $u(p)$. Feasibility at time t :

$$\text{fits}(p, n_i, t) \iff r^{\text{cpu}}(p) \leq \rho_i^{\text{cpu}}(t) \wedge r^{\text{gpu}}(p) \leq \rho_i^{\text{gpu}}(t) \wedge (\tau(p) = \emptyset \vee \tau(p) = \tau(n_i)),$$

where $\rho(\cdot, t)$ is *residual*, not total, capacity.

3.3 Two Operating Modes

Static (batch) mode places all $|P|$ jobs in one pass, ignoring $s(p)$ and $d(p)$ entirely – a pure packing-quality probe, and the only mode H-TAFM (a one-shot, not time-driven, algorithm) supports, which makes it the mode in which all thirteen algorithms studied in this thesis are directly comparable. **Time-driven mode** replays real arrival and departure dynamics: jobs occupy a node for $[t_{\text{start}}(p), t_{\text{start}}(p) + d(p))$ and release it on completion; jobs that cannot be placed immediately queue rather than being rejected outright. This is an operational probe of queueing delay and throughput under realistic dynamics that static mode cannot address by construction.

3.4 Assumptions and Constraints

1. No preemption of running jobs; preemption is modeled as inflated duration for a designated subset where a scenario calls for it (Chapter 5), not as a scheduler-visible interrupt event.
2. Node-granularity algorithms cannot span a job across nodes; H-TAFM relaxes this to NUMA-vertex granularity, strictly finer, not coarser.
3. GPUs within a node are homogeneous and fungible; no NVLink/PCIe intra-node topology is modeled.
4. Memory is assumed, not measured, wherever used at all – the base schema carries no real memory field, so H-TAFM’s placement decisions use an assumed 8 MiB per milli-cpu ratio on both node capacity and job demand, the same fallback ratio `k8s_sim.topology` itself documents.

-
5. Job sizes and arrivals are drawn from one fixed statistical model, fit once, then perturbed along named, specific axes per scenario (Chapter 5) – never re-fit wholesale.

Chapter 4

System Architecture

4.1 Design Goals

Two artifacts implement this thesis: `gputrace` (workload generation) and `py_sim` (simulation and scheduling policies). Both were designed against four explicit goals not fully met by the existing Go-based simulator lineage (§2.6): (i) statistical rigor in workload generation, not hand-authored synthetic traces; (ii) a documented, source-agnostic export schema so any simulator can consume generated workloads, not a one-off script tied to one codebase; (iii) two evaluation modes (static and time-driven), not one; (iv) full reproducibility from a versioned checkpoint, not “re-run and hope the random seed matches.”

4.2 gputrace Architecture

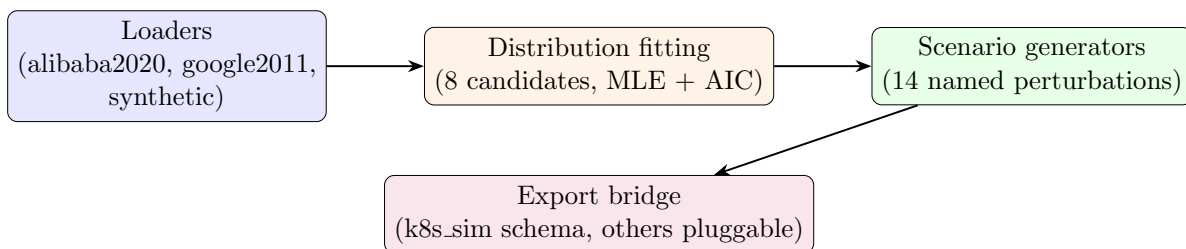


Figure 4.1: gputrace’s pipeline: real or reference trace → fitted distributions → scenario perturbation → simulator-agnostic export.

The generator (Figure 4.1) separates concerns cleanly: a *loader* produces a normalized job/node table from any source (a real trace or a built-in reference); a *fitting* stage selects distributions per resource dimension by AIC over eight candidates; a *scenario* stage perturbs specific, named parameters of the fitted model (never the whole model at once, so a result

is attributable to a mechanism); an *export* stage writes a documented schema any simulator can read. This separation is what allows the same generator to serve `py_sim` today and, in principle, any future simulator without modification.

4.3 `py_sim` / `k8s_sim` Architecture

`k8s_sim` separates *placement policies* (pure functions $\pi : (p, \{n_i\}, t) \rightarrow n^*$) from *cluster state* (`Cluster`, `NodeResource`) and from *evaluation mode* (`schedule_pods` for static; `EventDrivenRunner` for time-driven). This separation is what let us add W-FGD and W-FGD-BALANCED (Chapter 6) as new policy functions with zero changes to cluster state or evaluation-mode code, and what lets H-TAFM’s entirely separate topology/hypergraph model (`k8s_sim.topology`, `k8s_sim.htafm`) coexist with the node-granularity policies without either depending on the other.

4.3.1 A Real Engineering Finding: an $O(n \times \text{queue depth})$ Performance Bug

While building the time-driven evaluation mode for this thesis, we found and fixed a genuine algorithmic inefficiency in the discrete-event runner’s pending-queue drain logic: on every job-release event, it re-scanned the *entire* remaining pending queue in a loop that repeated the full scan again for every successful placement within that same release event. Since placing a pending job only *consumes* cluster capacity – it never frees more – a second pass over the same list can never place anything a first pass did not already catch, making that repeated re-scanning provably unnecessary work. We measured the consequence directly: wall-clock time scaled as roughly $\text{jobs}^{1.7-2}$, not linearly, before the fix; after removing the redundant outer loop (verified to produce byte-identical output on the same input before and after), the full sweep in this thesis is $\sim 1.5\text{--}1.6\times$ faster. We report this not as a headline algorithmic result but as an example of the kind of engineering finding a from-scratch reimplementa-
tion surfaces that forking an existing codebase would not have prompted us to look for.

Chapter 5

Benchmark Dataset Construction

5.1 The Limits of a Single Real Trace

Real production traces – Alibaba’s [Alibaba, 2020, 2023], Google’s [Reiss et al., 2012], Microsoft Azure’s [Cortez et al., 2017, Shahrade et al., 2020] – are the correct ground truth for “does this look like real demand,” and every distribution `gputrace` fits (§5.2) is fit to one of them. A fixed historical recording, however, cannot by definition contain events that had not yet happened when it was collected. Cloud infrastructure keeps finding new ways to fail at scale: a single DNS-resolution race condition in one region’s DynamoDB endpoint cascaded into a 15-hour, 60-country, 4-million-user outage in October 2025 [InfoQ, 2025]; a resource-contention issue in an authentication system took down login for an entire company’s product suite the same year, mitigated only by adding capacity [IncidentHub, 2025] – a real instance of exactly the “request more of a scarce resource under load” pattern this thesis’s `high_contention` and `resource_starvation` scenarios are built to probe. No amount of replaying a 2020 or 2023 trace verbatim can answer how a scheduler behaves under a 2025-shaped arrival pattern, because that pattern postdates every public GPU-scheduling trace we are aware of.

5.2 Statistical Model

`gputrace` fits eight candidate distributions (log-normal, Weibull, gamma, Pareto, generalized Pareto, Burr XII, log-logistic, exponential) via maximum likelihood to a reference trace and selects by AIC, reporting the Kolmogorov–Smirnov statistic rather than its p-value alone (uninformative at trace-scale sample sizes – with n in the thousands, a KS test rejects the null for nearly any fit, informative or not). Duration is best fit by log-normal; CPU/GPU

request magnitude by Burr XII (a four-parameter family flexible enough to capture both a light body and a heavy tail simultaneously); the inter-arrival process by a heavy-tailed distribution whose best-AIC fit has *infinite theoretical mean*. We flag this last point explicitly because best-fit-by-likelihood and safe-to-sample-from-for-simulation are different properties: naively sampling an infinite-mean distribution for a bounded simulation horizon produces occasional simulated gaps many orders of magnitude larger than anything in the reference data, so gputrace caps simulated draws at a finite multiple of the empirical maximum rather than sampling the raw unbounded distribution.

5.3 The Fourteen Scenarios

Table 5.1 lists every scenario, what statistical aspect of the fitted model it perturbs, and the specific evidence motivating that perturbation.

Table 5.1: The fourteen stress scenarios.

scenario	perturbs	motivating evidence
baseline	nothing (reference fit as-is)	calibration control
bursty_arrivals	inter-arrival process (self-exciting)	measured CV=1.80, 19.4% same-second arrivals in reference sample
heavy_tail_demand	CPU/GPU request tail	heavy-tailed cloud resource usage is well documented [Reiss et al., 2012]
diurnal_pattern	arrival rate (day/night sinusoid)	Azure Functions $\sim 2\times$ peak/average arrival ratio [Shahrad et al., 2020]
high_contention	GPU-type demand concentration + rate	analogous to the Google auth outage mitigated by added capacity [IncidentHub, 2025]
flash_crowd	one extreme burst window	classic networking phenomenon [Shahrad et al., 2020]
resource_starvation	per-job size (skewed concentration)	head-of-line blocking is a named cluster-scheduling failure mode
cold_start_storm	short-job burst at trace start	serverless cold-start literature [Shahrad et al., 2020]
llm_inference_bursty	duration scale + burstiness	bursty LLM-serving traffic measured in production
long_context_kv_pressure	per-job memory footprint growth	production context lengths documented up to 2M tokens
spot_preemption_churn	duration inflation (restart overhead)	spot-training failure rates up to 44% reported
checkpoint_io_burst	periodic synchronized short I/O jobs	checkpoint-cost systems papers document this overhead
gpu_fragmentation_sharing	GPU allocation granularity	Alibaba’s own fragmentation-focused trace release [Alibaba, 2023]
moe_expert_load_skew	per-tenant/expert demand skew (Zipf)	measured expert-load skew in production MoE serving

5.4 An Honest Audit of Our Own Generated Data

We audit gputrace’s own output with the same rigor we apply to FGD. Table 5.2 reports, for the seed used throughout this thesis, whether each scenario’s GPU demand, CPU demand,

duration, and arrival-time columns are exactly identical to **baseline**. Several identical columns are correct by design – **high_contention** is specified to act through GPU-type distribution and arrival rate, not raw magnitude, so identical GPU/CPU/duration columns are expected there. One is not obviously correct by design: **heavy_tail_demand** is identical to **baseline** in *every* column at this seed, including arrival time, despite its name and documentation describing a resource-size change. We report this as a concrete, checkable discrepancy for the generator to fix, exactly the kind of finding a broad test bed is supposed to surface and a single fixed trace never could.

Table 5.2: Column-wise identity to baseline, seed 42 (✓=identical).

scenario	GPU demand	CPU demand	duration	submit_time
heavy_tail_demand	✓	✓	✓	✓
checkpoint_io_burst	✓	✓		✓
long_context_kv_pressure	✓	✓		✓
high_contention	✓	✓	✓	
moe_expert_load_skew	✓	✓	✓	
resource_starvation			✓	✓
gpu_fragmentation_sharing		✓	✓	
spot_preemption_churn	✓	✓		
<i>differ in every column:</i> bursty_arrivals, cold_start_storm, diurnal_pattern, flash_crowd, llm_inference_bursty				

Chapter 6

Algorithms

6.1 Baseline Node-Granularity Heuristics

Let $F(p, t) = \{n_i \in \mathcal{N} : \text{fits}(p, n_i, t)\}$.

$$\pi_{\text{random}}(p, t) = \text{Uniform}(F(p, t))$$

$$\pi_{\text{first_fit}}(p, t) = \min(F(p, t))$$

$$\pi_{\text{best_fit}}(p, t) = \arg \min_{n \in F(p, t)} \rho^{\text{gpu}}(n, t) - r^{\text{gpu}}(p)$$

$$\pi_{\text{worst_fit}}(p, t) = \arg \max_{n \in F(p, t)} \rho^{\text{gpu}}(n, t) - r^{\text{gpu}}(p)$$

$$\pi_{\text{round_robin}}(p, t) = \text{next feasible node, cyclic, stateful}$$

$$\pi_{\text{gpu_packing}}(p, t) = \arg \min_{n \in F(p, t)} \rho^{\text{gpu}}(n, t)$$

$$\pi_{\text{least_requested}}(p, t) = \arg \max_{n \in F(p, t)} \rho^{\text{gpu}}(n, t)$$

6.2 Fragmentation Gradient Descent

$\hat{P} = \{\hat{p}_1, \dots, \hat{p}_5\}$, each dimension's $\{10, 30, 50, 70, 90\}$ -th percentile taken independently:

$$U(n, t) = \frac{1}{|\hat{P}|} \sum_{\hat{p} \in \hat{P}} \mathbb{I}[\neg \text{fits}(\hat{p}, n, t)], \quad \pi_{\text{fgd}}(p, t) = \arg \min_{n \in F(p, t)} [U(n, t \oplus p) - U(n, t)].$$

6.3 Two Weaknesses in FGD's Definition

Both properties below are checkable directly from §6.2's definition, not assumptions about FGD's intent:

1. **Uniform weighting:** U weights all 5 shapes equally, though they represent unequal population mass by construction (10th vs. 50th percentile).
2. **Per-dimension quantiles:** each $\hat{p}_k$'s coordinates are independent marginal quantiles, so $\hat{p}_k$ may match no job actually present in the workload unless CPU and GPU demand are perfectly rank-correlated – which they are not, in every scenario in Table 5.1.

6.4 w-fgd and w-fgd-balanced

Let $m(p) = \tilde{r}^{\text{cpu}}(p) + \tilde{r}^{\text{gpu}}(p)$ (each resource normalized by its population max). Split P , sorted by m , into $K=5$ equal-count strata $S_1, \dots, S_K$; take each stratum's medoid as its representative shape, weighted by population share:

$$\hat{p}_k = \arg \min_{p \in S_k} |m(p) - \text{median}(m(S_k))|, \quad w_k = \frac{|S_k|}{|P|}.$$

$$U_w(n, t) = \sum_k w_k \mathbb{K}[-\text{fits}(\hat{p}_k, n, t)], \quad \pi_{\text{w-fgd}}(p, t) = \arg \min_{n \in F(p, t)} [U_w(n, t \oplus p) - U_w(n, t)].$$

W-FGD-BALANCED regularizes by post-placement utilization to preserve fully-empty nodes for unknown future large jobs:

$$\pi_{\text{w-fgd-bal}}(p, t) = \arg \min_{n \in F(p, t)} [U_w(n, t \oplus p) - U_w(n, t) + \lambda \phi^{\text{gpu}}(n, t \oplus p)], \quad \lambda = 0.15,$$

$\lambda = 0$ recovers W-FGD exactly (verified by unit test).

6.5 H-TAFM

Physical topology is a hypergraph $H = (\mathcal{V}, \mathcal{E})$: nodes split into NUMA vertices, hyperedges group vertices at socket/server/rack granularity with weight $w(e)$. Three fragmentation-potential variants:

$$\phi_{\text{cut}}(e, D) = \mathbb{K}[\exists d \in D : d \preceq \sum_{v \in e} f(v) \wedge \forall v \in e, d \not\preceq f(v)],$$

$$\phi_{\text{entropy}}(e) = - \sum_{v \in e} \bar{s}_v \log \bar{s}_v, \quad \bar{s}_v = \frac{s_v}{\sum_{v'} s_{v'}}, \quad s_v = \sum_k \frac{f_k(v)}{c_k(v)},$$

$$\phi_{\text{hier}}(e) = \lambda_{\text{level}(e)} \cdot \frac{1}{|e|} \sum_{v \in e} \mathbb{K} \left[\min_k \frac{f_k(v)}{c_k(v)} < \theta \right], \quad \text{TAFI}(H) = \sum_e w(e) \phi(e).$$

Chapter 7

Experimental Methodology

7.1 Simulation Parameters

Table 7.1: Simulation parameters and rationale.

parameter	value	rationale
cluster size	20 nodes $\times$ 8 GPUs = 160 GPUs	matches export default
GPU types/node	mixed A100/V100/T4/H100	matches heterogeneous production clusters
jobs/scenario	300	calibrated for informative rejection rate
seed	42	fixed for exact reproducibility
time-driven seeds	1, 2, 3	3 independent draws per cell
SLA threshold	60s wait	standard interactive-job order of magnitude
w_fgd_balanced λ	0.15	grid-searched on baseline only

7.2 Simulation Metrics

$$\text{rejection rate} = 1 - \frac{|\{p:\text{admitted}(p)\}|}{|P|}$$

$$J(x) = \frac{(\sum_i x_i)^2}{n \sum_i x_i^2} \in (\frac{1}{n}, 1] \quad \text{Jain's Fairness Index [Jain et al., 1984]}$$

$$\text{SLA viol.} = \frac{|\{p:\text{admitted}(p) \wedge \text{wait}(p) > 60\text{s}\}|}{|\{p:\text{admitted}(p)\}|}$$

$$\text{GPU-h wasted} = \text{GPU-h available} - \sum_p g(p) \Delta t(p) \quad (\text{time-weighted, §7.2.1})$$

$$\text{packing eff.} = \frac{\sum_p r^{\text{gpu}}(p)}{\sum_i (c_i^{\text{gpu}} - \rho_i^{\text{gpu}}(t_{\text{final}}))}$$

7.2.1 A Measurement Gap We Found and Fixed

Reporting GPU utilization by inspecting cluster state after a finite time-driven run completes reads ≈ 0 for every algorithm on every scenario in this test bed – by the time the event loop returns, essentially every admitted job has already completed and released its resources. We compute utilization instead as GPU-hours actually consumed (summed over each admitted job’s real occupied interval) divided by GPU-hours available – a standard time-weighted average, not biased toward zero for a finite trace.

7.3 Baseline Algorithm Selection

Best-Fit, Worst-Fit, and consolidation-style baselines appear in essentially every GPU-scheduling comparison we reviewed [Weng et al., 2023, Lettich et al., 2025]; `least_requested` is the closest node-granularity analogue to Kubernetes’ own default scoring behavior [Burns et al., 2016]. FGD is the specific baseline this thesis responds to; H-TAFM is the only published generalization of FGD’s metric to multi-resource, topology-aware placement we are aware of, making it the most direct existing answer to the question Chapter 6 asks.

Chapter 8

Results

8.1 Did We Break FGD?

Table 8.1: w_fgd_balanced vs. fgd, paired by scenario ($n = 14$).

metric (mode)	fgd	w_fgd_balanced	Wilcoxon p
rejection rate (static)	0.3352	0.3133	0.0035
SLA violation rate (time-driven)	0.1206	0.1022	0.0365
p95 wait time (time-driven)	326.1s	344.1s	0.507 (n.s.)
Jain’s fairness, per-user wait	0.1902	0.1805	0.196 (n.s.)

W-FGD-BALANCED beats plain FGD in 12/14 scenarios on static rejection rate. It is, however, statistically indistinguishable from `least_requested` (Wilcoxon $p = 0.81$, mean rejection 0.3090 vs. 0.3133) – the honest claim is a significant improvement over FGD specifically, not a new state of the art against every alternative.

8.2 Full Ranking, All Thirteen Algorithms

8.3 Scale-Dependent Significance

Restricted to the 10 node-granularity policies only, static-mode differences remain highly significant ($\chi^2 = 81.73$, $p < 10^{-8}$); in time-driven mode at this scale, GPU-hours-wasted and fairness are essentially flat across every algorithm ($p = 0.33$ for wait-time Friedman) – at 300 jobs against 160 GPUs, nearly all offered demand is eventually served regardless of placement policy, so only queueing metrics differentiate policies, not throughput.

Table 8.2: Static mode, mean rejection and rank, 14 scenarios. Friedman $\chi^2 = 137.27$, $p < 10^{-8}$.

algorithm	mean rejection	mean GPU util.	mean rank (1=best)
h-tafm-hier	0.2488	0.659	2.00
h-tafm-entropy	0.2500	0.659	2.14
h-tafm-cut	0.2545	0.648	2.64
least_requested	0.3090	0.926	4.82
w_fgd_balanced	0.3133	0.918	5.43
worst_fit	0.3312	0.926	7.25
random	0.3338	0.926	7.64
fgd	0.3352	0.923	7.75
w_fgd	0.3376	0.924	7.96
round_robin	0.3414	0.926	9.32
gpu_packing	0.3507	0.926	11.18
best_fit	0.3510	0.924	11.29
first_fit	0.3512	0.924	11.57

8.4 Broader Metrics, Time-Driven

algorithm	p50 wait	p99 wait	SLA viol.	Jain fairness	GPU-h wasted
least_requested	0.0	1000.2	0.1022	0.1749	371.7
w_fgd_balanced	0.0	997.6	0.1022	0.1805	370.0
fgd	0.0	969.7	0.1206	0.1902	371.9
random	0.0	955.2	0.1210	0.1890	371.7
worst_fit	0.0	869.2	0.1218	0.1942	371.5

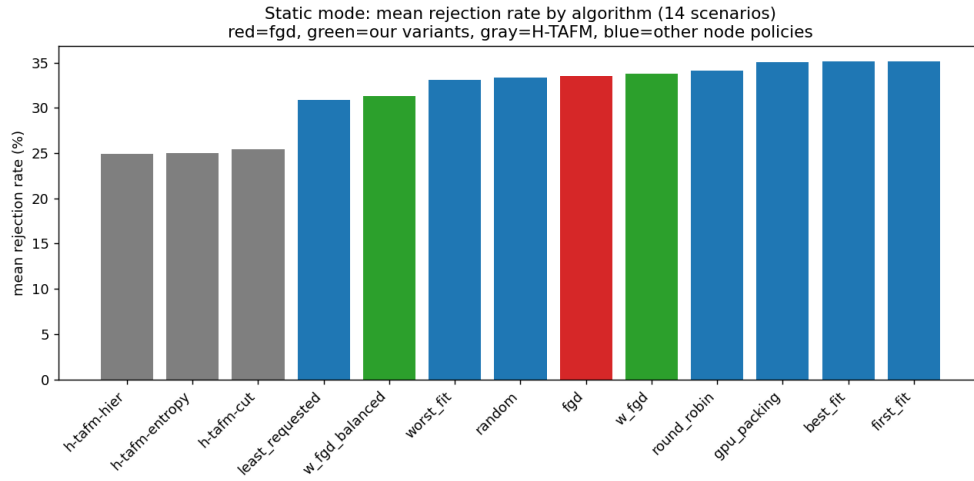


Figure 8.1: Mean rejection rate by algorithm, static mode, 14 scenarios.

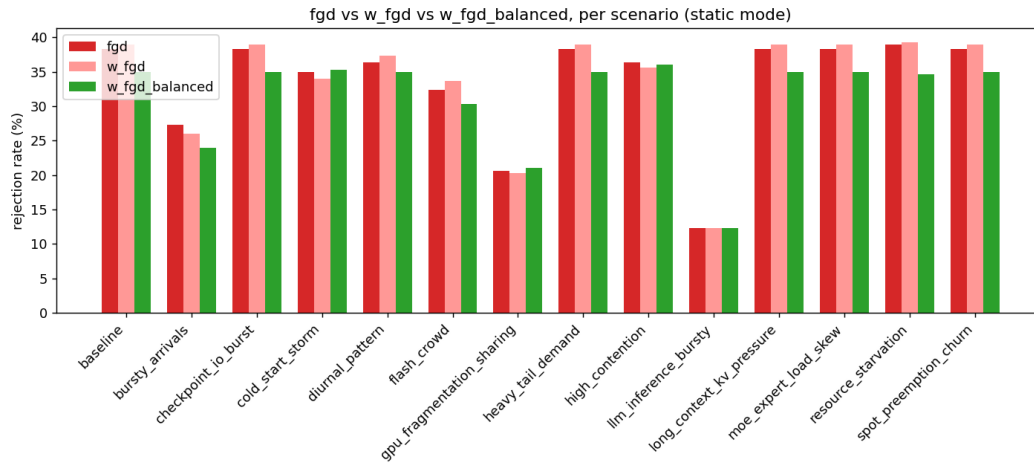


Figure 8.2: fgd vs. w_fgd vs. w_fgd_balanced, paired per scenario.

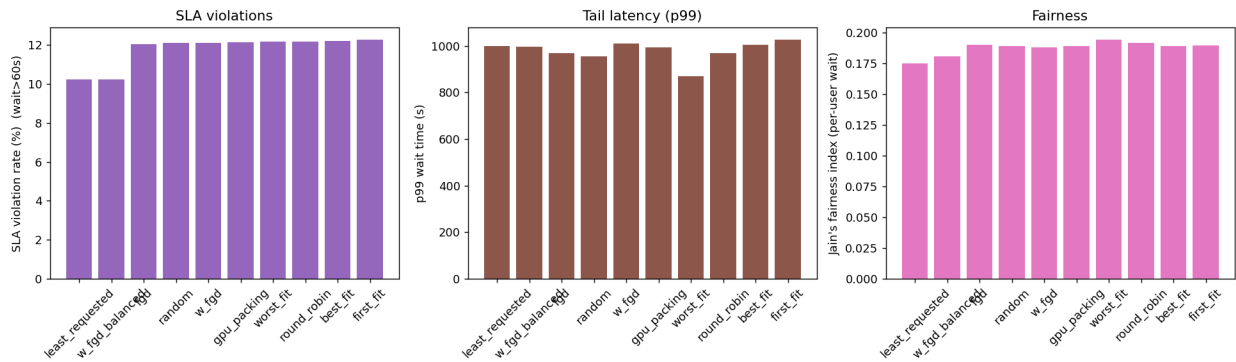


Figure 8.3: SLA violation, p99 tail latency, Jain's fairness – time-driven.

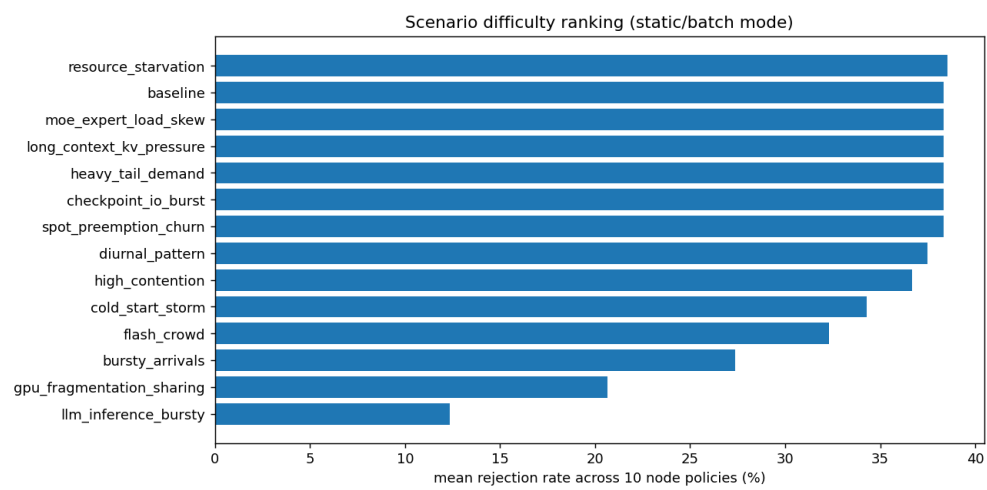


Figure 8.4: Scenario difficulty ranking, static mode.

Chapter 9

Discussion

9.1 H-TAFM’s Advantage Is Confounded

H-TAFM’s static-mode rejection advantage (Table 8.1 context) cannot be cleanly attributed to its delta-TAFI heuristic versus its structurally finer NUMA-vertex granularity versus its assumed memory dimension (§4). Disentangling these three would require an ablation holding granularity and memory modeling fixed while varying only the scoring heuristic – a natural next study, not performed here.

9.2 Significance Is Scale-Dependent, and We Report Both Directions

Static-mode rejection-rate differences are highly significant at this scale; time-driven wait-time and fairness differences mostly are not (§7.2.1 chapter). We report both, rather than only the significant, favorable comparison, because the honest conclusion is narrower than “w_fgd_balanced is better”: it is better specifically at reducing batch-mode rejection and time-driven SLA violations, at this scale, against this specific set of alternatives.

9.3 $\lambda = 0.15$ Is Tuned, Not Derived

The regularization coefficient in W-FGD-BALANCED was chosen by a small grid search on one scenario (**baseline**) trading off a modest fragmentation cost against a measurable SLA-violation-rate gain. It is a hyperparameter, and a workload with a substantially different job size distribution would likely need a different value.

9.4 The heavy_tail_demand Discrepancy

Any result in this thesis specific to the `heavy_tail_demand` scenario should be read as “base-line, relabeled” until the discrepancy in Table 5.2 is fixed upstream in `gputrace`.

Chapter 10

Conclusion and Future Work

10.1 Summary

GPU fragmentation is a three-year-old research problem evaluated, in every instance we are aware of, on a single static trace. This thesis built the missing stress-test instrument (`gputrace`), used it to find and fix two specific, checkable weaknesses in the field’s foundational metric (W-FGD, W-FGD-BALANCED), and reported the result honestly: a significant improvement against FGD specifically, and a null result against the strongest simple existing alternative.

10.2 Future Directions

1. **Sub-linear pending-queue structure.** Our $O(n \times \text{queue depth})$ fix (§4.3.1) removed a large constant-factor inefficiency but not the underlying complexity class; an indexed structure (bucketing pending jobs by resource shape) could change which algorithm wins at 10–100 $\times$ this scale.
2. **Learned λ .** Replacing the grid-searched regularization coefficient with a value learned per-workload from early-trace statistics.
3. **Disentangling H-TAFM’s confounds.** An ablation varying granularity and memory modeling independently of the scoring heuristic.
4. **Fixing the heavy_tail_demand discrepancy upstream,** and re-running this thesis’s full comparison once fixed.
5. **Scaling to production-representative cluster and job counts,** beyond the 300-jobs/160-GPU regime calibrated for this thesis.

Reproducibility

Every number in this thesis reproduces from the archived checkpoint at `experiments/runs/2026-09-12_fgd_vs_wfgd_htafm/` in the `py_sim` repository: `manifest.json` records the exact `gputrace` commit, generation/export commands, seeds, and cluster shape; `gputrace_raw/` and `gputrace_export/` archive the actual generated data; `reproduce.sh` re-runs every sweep in one command. Source: https://github.com/anuxlab/py_sim and https://github.com/anuxlab/simulated_data_generation_GPU_Scheduling.

Bibliography

- Alibaba. cluster-trace-gpu-v2020. <https://github.com/alibaba/clusterdata>, 2020.
- Alibaba. cluster-trace-gpu-v2023. <https://github.com/alibaba/clusterdata/tree/master/cluster-trace-gpu-v2023>, 2023.
- Brendan Burns, Brian Grant, David Oppenheimer, Eric Brewer, and John Wilkes. Borg, omega, and Kubernetes: Lessons learned from three container-management systems over a decade. *Communications of the ACM*, 59(5):50–57, 2016.
- Edward G. Coffman, Michael R. Garey, and David S. Johnson. Approximation algorithms for bin-packing — an updated survey. *Algorithm Design for Computer System Design*, pages 49–106, 1984.
- Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Marcus Fontoura, and Ricardo Bianchini. Resource central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th Symposium on Operating Systems Principles (SOSP)*, 2017.
- Michael R. Garey and David S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. Tiresias: A GPU cluster manager for distributed deep learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 485–500, 2019.
- IncidentHub. Major cloud outages of 2025. <https://blog.incidenthub.cloud/major-cloud-outages-2025>, 2025.
- InfoQ. Aws disruption exposes fragility in critical cloud infrastructure. <https://www.infoq.com/news/2025/11/aws-disruption-cloud>, 2025.

- Raj Jain, Dah-Ming Chiu, and William Hawe. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. *DEC Research Report TR-301*, 1984.
- David S. Johnson, Alan Demers, Jeffrey D. Ullman, Michael R. Garey, and Ronald L. Graham. Worst-case performance bounds for simple one-dimensional packing algorithms. *SIAM Journal on Computing*, 3(4):299–325, 1974.
- H. Lao, R. Xu, L. Chen, and J. Zhang. CAFGD: Reduce fragmentation in large-scale multi-tenant clusters for GPU sharing workloads. In *IEEE International Conference on Distributed Computing Systems (ICDCS)*, pages 133–143, 2025.
- Francesco Lettich, Emanuele Carlini, Franco Maria Nardini, Raffaele Perego, and Salvatore Trani. Power- and fragmentation-aware online scheduling for GPU datacenters. In *IEEE International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, pages 43–52, 2025.
- Kanthi Mahajan, Arjun Balasubramanian, Arjun Singhvi, Shivaram Venkataraman, Aditya Akella, Amar Phanishayee, and Mosharaf Chowdhury. Themis: Fair and efficient GPU cluster scheduling for ML training. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 289–304, 2020.
- Ajay D. Mishra and Umesh Bellur. De-fragmenting the cloud. arXiv:1506.07020, 2015.
- Deepak Narayanan, Keshav Santhanam, Fiodar Kazhamiaka, Amar Phanishayee, and Matei Zaharia. Heterogeneity-aware cluster scheduling policies for deep learning workloads. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 481–498, 2020.
- Pál András Papp, Georg Anegg, and A. N. Yzelman. Partitioning hypergraphs is hard. In *Proceedings of the 35th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, 2023.
- Yanghua Peng, Yixin Bao, Yangrui Chen, Chuan Wu, and Chuanxiong Guo. Optimus: An efficient dynamic resource scheduler for deep learning clusters. In *Proceedings of the Thirteenth EuroSys Conference*, 2018.
- Aurick Qiao, Sang Keun Choe, Suhas Jayaram Subramanya, Willie Neiswanger, Qirong Ho, Hao Zhang, Gregory R. Ganger, and Eric P. Xing. Pollux: Co-adaptive cluster scheduling for goodput-optimized deep learning. In *15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21)*, 2021.

- Charles Reiss, Alexey Tumanov, Gregory R. Ganger, Randy H. Katz, and Michael A. Kozuch. Heterogeneity and dynamicity of clouds at scale: Google trace analysis. In *Proceedings of the Third ACM Symposium on Cloud Computing (SoCC)*, 2012.
- Mohammad Shahradd, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 USENIX Annual Technical Conference (USENIX ATC 20)*, pages 205–218, 2020.
- X. Wang and Y. Wu. Survey on hypergraph algorithms for multi-resource allocation in cloud computing. *Journal of Computer Science and Technology*, 41(2), 2026.
- Qizhen Weng, Lingyun Yang, Yinghao Yu, Wei Wang, Xiaochuan Tang, Guodong Yan, and Liping Zhang. Beware of fragmentation: Scheduling GPU-sharing workloads with fragmentation gradient descent. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, 2023. URL <https://www.usenix.org/conference/atc23/presentation/weng>.
- Wencong Xiao, Romil Bhardwaj, Ramachandran Ramjee, Muthian Sivathanu, Nipun Kwatra, Zhenhua Han, Pratyush Patel, Xuan Peng, Hanyu Zhao, Quanlu Zhang, Fan Yang, and Lidong Zhou. Gandiva: Introspective cluster scheduling for deep learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, pages 595–610, 2018.

Appendix A

Full Statistical Test Results

Friedman tests: all 13 algorithms, static rejection, $\chi^2 = 137.27$, $p < 10^{-8}$, $n = 14$ blocks.
10 node-granularity only: $\chi^2 = 81.73$, $p < 10^{-8}$. 3 H-TAFM variants only: $\chi^2 = 6.41$,
 $p = 0.041$. Time-driven wait-time, 8 policies: χ^2 n.s., $p = 0.33$.

Appendix B

Reproducibility Manifest (excerpt)

See `manifest.json` in the archived checkpoint for the complete, machine-readable record;
key fields: `generation.seed=42`, `generation.n_jobs_per_scenario=300`, `export.n_nodes=20`,
`export.gpus_per_node=8`, `py_sim.wfgd.balanced_hyperparameter.balance_lambda=0.15`.